



ZLAARK PYTHON INTERNSHIP • SESSION 15 OF 25

Recap & Doubt Clearing.

Seven days of learning. One day to consolidate. Today we find what's shaky, fix it, and verify it works. Tomorrow the group project begins.

PROGRAM
Python Internship



LED BY
Zlaark



SESSION
15 of 25



Phase 3 • Recap

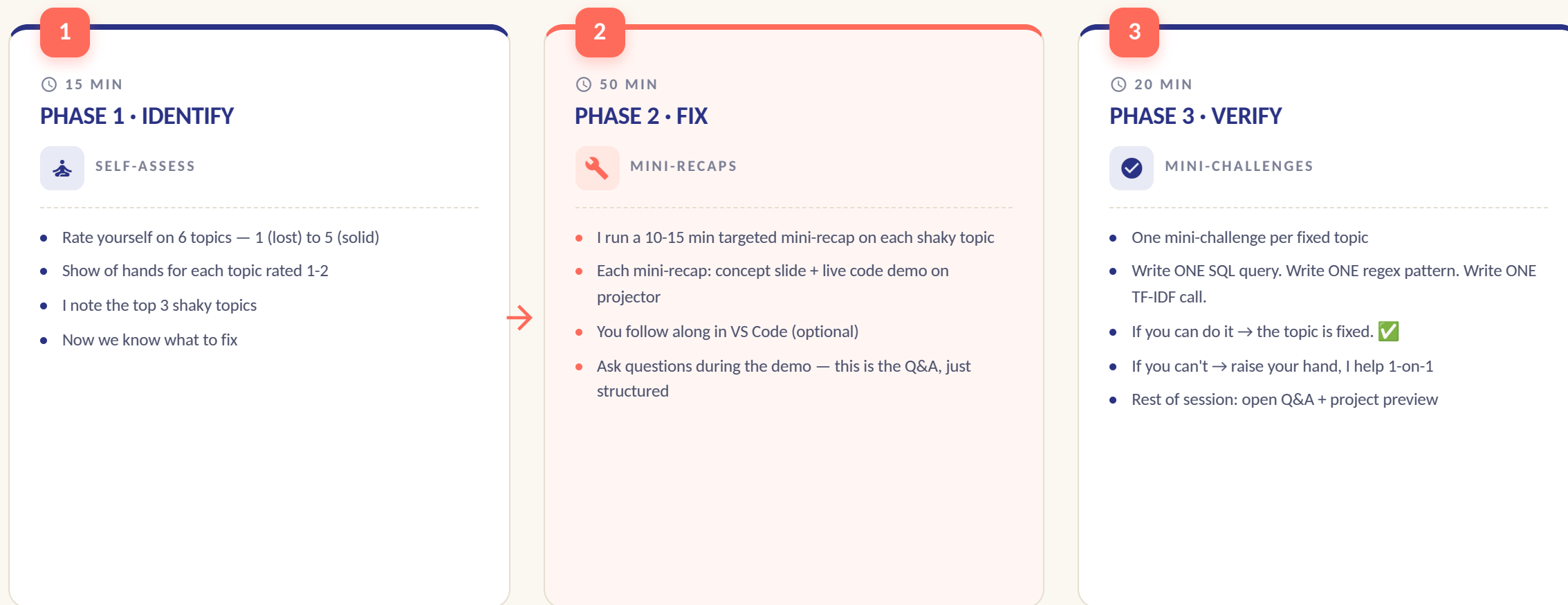
Doubt Clearing

DATE

11 August 2026

Identify what's shaky. Fix it. Verify it works.

No open-ended Q&A where nobody talks. Instead: you rate yourself, I review the top shaky topics, you prove you can do it. Three phases, 90 minutes.



THE STRUCTURE — You drive the content (what's shaky). I drive the format (identify → fix → verify). No silence, no wasted time. By the end, you KNOW what you can do and what needs work before tomorrow.

• PHASE 1 • IDENTIFY (15 MIN)

Rate yourself. Be honest. No judgment.

📁 6 topics • 1-5 scale

For each topic, rate yourself 1-5. 1 = I'm lost. 5 = I could teach it. Then we'll do a show of hands for each topic rated 1-2. This drives the rest of the session.

SQL & SQLite

📅 DAYS 9-10

`CREATE TABLE`, `INSERT`, `SELECT`, `WHERE`, `JOIN`

The `with` block: commits transaction, does NOT close.

RATING ○ ○ ○ ○ ○

Regex

📅 DAY 11

`re.findall`, `re.search`, `re.sub`

Escaping (`\.`), greedy vs lazy (`.*` vs `.*?`), flags.

RATING ○ ○ ○ ○ ○

PDF & Text Extraction

📅 DAY 11

PyPDF2: `PdfReader`, `extract_text()`

python-docx: paragraphs, tables. Text cleaning: regex pipeline.

RATING ○ ○ ○ ○ ○

APIs & requests

📅 DAY 12

`requests.get`, `response.json()`, `params` dict.

Errors: timeout, raise_for_status, try/except. `@st.cache_data`.

RATING ○ ○ ○ ○ ○

TF-IDF & AI

📅 DAY 13

`TfidfVectorizer(stop_words='english')`

`cosine_similarity` → score 0 to 1. Limits: synonyms, context, order.

RATING ○ ○ ○ ○ ○

Streamlit & UI

📅 DAYS 4-5, 7

Widgets: `file_uploader`, `text_area`, `metric`, `columns`

Layout: sidebar, tabs, expander. Charts: `st.bar_chart`, `st.dataframe`.

RATING ○ ○ ○ ○ ○



SHOW OF HANDS

For each topic, raise your hand if you rated yourself 1 or 2. I'll count and note the top 3 shaky topics. Those are what we'll fix in Phase 2.

BE HONEST — 2/5 is normal for AI topics (TF-IDF). 1/5 on SQL means we need to review the helper-function pattern. The point is to find the gaps, not to feel bad about them. Tomorrow you'll be in groups — your teammates fill your gaps.

SQL & SQLite — the 4-step pattern you'll use in the group project.

If you rated SQL 1-2, this is for you. The whole pattern in one slide: connect → execute → fetch → close. Plus the with-block gotcha and the helper-function pattern.

• • • sql_refresher.py

```
import sqlite3

# === THE 4-STEP PATTERN ===
# Step 1: Connect
conn = sqlite3.connect("app.db")
cursor = conn.cursor()

# Step 2: Execute (CREATE / INSERT / SELECT)
cursor.execute("""
    CREATE TABLE IF NOT EXISTS expenses (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        amount REAL NOT NULL,
        category TEXT DEFAULT 'Other'
    )
""")

# Step 3: Commit (for writes) or Fetch (for reads)
cursor.execute("INSERT INTO expenses VALUES (NULL, 450, 'Food')", )
conn.commit() # ← DON'T FORGET!

cursor.execute("SELECT * FROM expenses")
rows = cursor.fetchall() # ← returns list of tuples

# Step 4: Close
conn.close()

# === THE with BLOCK GOTCHA ===
# `with sqlite3.connect(...) as conn:`
#   commits on success, rolls back on error
#   → does NOT close the connection!
#   → use conn.close() after. OR:
```

❗ 3 COMMON MISTAKES

The silent killers in SQL code.

✗ Forgot `conn.commit()`

→ Data disappears on restart. No error — silent failure.

✓ Fix: Always `commit()` after INSERT/UPDATE/DELETE.

✗ Using f-strings for SQL values

→ SQL injection risk. Also breaks on quotes/special chars.

✓ Fix: Use `?` placeholders. `cursor.execute("... VALUES (?, ?)", (val1, val2))`

✗ Confused about with closing

→ `with conn:` commits but doesn't close.

✓ Fix: Use `with closing(sqlite3.connect(...)) as conn:` for auto-close.
OR just call `conn.close()` explicitly — the helper-function pattern works fine.

Regex — the 3 functions + the 2 gotchas.

If you rated regex 1-2, this is for you. Three functions (findall, search, sub), two gotchas (escaping, greedy/lazy), and the patterns you'll reuse.

Σ THE 3 FUNCTIONS

```
import re

# 1. FINDALL — get all matches (returns a list)
emails = re.findall(r'\w.+-\w+@\w-\w+\.\w.+', text)

# 2. SEARCH — get first match (returns Match or None)
match = re.search(r'\d{10}', text)
if match:
    phone = match.group() # use .group()

# 3. SUB — replace matches (returns modified string)
cleaned = re.sub(r'\s+', ' ', text) # normalize whitespace
masked = re.sub(r'\d{10}', 'XXXXXXXXXX', text) # redact phones
```

Always use `r"..."` (raw strings) so `\d`, `\w`, `\s` work correctly.

⚠ THE 2 GOTCHAS

1 ESCAPING

`.` matches ANY character. To match a literal dot: `\.`

```
re.findall(r'.', 'a.b.c') # → ['a', '.', 'b', '.', 'c'] (every char!)
re.findall(r'\.', 'a.b.c') # → ['.', '.'] (just the dots)
```

Escape: `.` → `\.` `*` → `\*` `+` → `\+` `?` → `\?` `(` → `\(`

2 GREEDY vs LAZY

`.*` matches as MUCH as possible. `.+?` matches as LITTLE as possible.

```
text = '<b>Hello</b> <b>World</b>'
re.findall(r'<b>.*</b>', text) # → ['<b>Hello</b> <b>World</b>'] (ONE match, too greedy)
re.findall(r'<b>.*?</b>', text) # → ['<b>Hello</b>', '<b>World</b>'] (TWO matches, lazy)
```

THE GROUP PROJECT CONNECTION — 4 of 10 projects need regex: resume matcher (extract email/phone from PDF), plagiarism checker (split text into sentences), FAQ assistant (clean user input), notes summarizer (split into sentences). The email pattern alone solves half the resume matcher.

TF-IDF — it's the 5-line pattern. Not magic.

If you rated TF-IDF 1-2, this is for you. The intuition, the code, and what it can't do. By the end of this slide, the "magic" should feel like math.

THE INTUITION + CODE

INTUITION — 3 STEPS

- 1 Turn text into numbers (TF-IDF vectors — important words get high weights)
- 2 Compare the numbers (cosine similarity — 0 to 1)
- 3 The score tells you how similar the texts are

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 5 lines. That's the whole technique.
vectorizer = TfidfVectorizer(stop_words='english')
vectors = vectorizer.fit_transform([resume, job_desc])
score = cosine_similarity(vectors[0:1], vectors[1:2])[0][0]
print(f"Match: {score*100:.0f}%")

# What stop_words='english' does:
# Removes 'the', 'is', 'a', 'and' — common words that carry no meaning.
# Without it, every document looks similar (they all have 'the').
```

WHAT IT CAN'T DO

4 LIMITATIONS — ONE LINE EACH

- ✗ **Synonyms:** 'ML' ≠ 'machine learning' (different words to TF-IDF)
- ✗ **Word relationships:** 'developer' ≠ 'development' (no stemming)
- ✗ **Context:** 'river bank' = 'money bank' (same word, one score)
- ✗ **Word order:** 'dog bites man' = 'man bites dog' (same words)

WHEN TO USE TF-IDF

Keyword matching, document similarity, quick prototyping. For the group project, it's **ENOUGH**. For production AI, upgrade to BERT/GPT embeddings.

THE GROUP PROJECT CONNECTION

4 of 10 projects use TF-IDF: resume matcher (Day 14's app), plagiarism checker, notes summarizer, FAQ assistant. The 5-line pattern is the foundation for all of them.

APIs + Streamlit — the 3 patterns you'll reuse.

Only if the show of hands showed this is shaky. Otherwise, skip to Phase 3.

API CALL PATTERN

The universal call: GET with params, timeout, error check, parse JSON.

```
import requests
response = requests.get(url, params={...}, timeout=10)
response.raise_for_status()
data = response.json()
```

Always: timeout=, raise_for_status(), try/except. The 3 rules.

CACHING PATTERN

Cache API results so reruns don't hit the rate limit.

```
@st.cache_data(ttl=600)
def get_data(params):
    response = requests.get(url, params=params, timeout=10)
    return response.json()
```

Without **@st.cache_data**, EVERY rerun calls the API. Rate limit = 429.

STREAMLIT UI PATTERN

Input → button → process → display. The universal flow.

```
st.title("My App")
user_input = st.text_input("Enter something", key="input")
if st.button("Submit", key="submit"):
    result = process(user_input)
    st.metric("Result", result)
```

The universal pattern: input widget → button → process → display.

IF THIS ISN'T SHAKY

Skip to Phase 3. The mini-challenges are next. If only 2-3 students rated APIs as shaky, help them 1-on-1 during the group project.

• PHASE 3 • VERIFY (20 MIN)

One challenge per topic. Can you do it?

Not graded. Not a test. Just a checkpoint. If you can do these, you're ready for the group project. If you can't, raise your hand — I help 1-on-1.



SQL CHALLENGE

⌚ 5 MIN

CHALLENGE

Write a Python function that connects to `test.db`, creates a table called `users` (id, name, email), inserts one row, and selects all rows.

HINT

Use the 4-step pattern: connect → execute CREATE → execute INSERT → commit → execute SELECT → fetchall → close

EXPECTED

~8 lines of Python. Uses ? placeholders. Calls `commit()`.



REGEX CHALLENGE

⌚ 3 MIN

CHALLENGE

Write a regex that finds ALL email addresses in this text: "Contact `aarav@gmail.com` or `priya@yahoo.co.in`"

HINT

Pattern: `[\w.+]+@[\w-]+\.[\w.]+` — from the Day 11 cheatsheet

EXPECTED

`re.findall(r'[\w.+]+@[\w-]+\.[\w.]+', text)` → returns both emails



TF-IDF CHALLENGE

⌚ 5 MIN

CHALLENGE

Write 5 lines of Python that compute the cosine similarity between two texts using TF-IDF.

HINT

`TfidfVectorizer(stop_words="english")` → `fit_transform([t1, t2])` → `cosine_similarity(v[0:1], v[1:2])[0][0]`



STREAMLIT CHALLENGE

⌚ 3 MIN

CHALLENGE

Write a 5-line Streamlit app that takes a text input and shows it back when a button is clicked.

HINT

`st.title` → `st.text_input(keyed)` → `st.button(keyed)` → if button: `st.write(input)`

IF YOU CAN DO ALL 4

You're ready for the group project. If you can do 3 — you're ready, the 4th will solidify in the project. If you can do fewer than 3 — raise your hand, we fix it 1-on-1 now, not during the project.

Tomorrow you start building. Together.

Day 16: groups formed, projects picked, sprints begin. Seven days of building. Five sprints. Demo Day on Day 25.

THE 7-DAY SPRINT

DAYS 16-22

Day 16 Kickoff & Groups
Groups of 4-5 announced. Project brief shared. Pick a project.

Day 17 Sprint 1: Skeleton
Basic Streamlit app + navigation. Something that runs.

Day 18 Sprint 2: Core Logic
The main feature. Real logic behind it.

Day 19 Sprint 3: Data
Save/load data (SQLite or CSV). Test with real inputs.

Day 20 Sprint 4: Polish
Clean UI, handle bad input, fix rough edges.

Day 21 Sprint 5: Feature Freeze
No new features. Full run-through. Fix what breaks.

Day 22 Deployment
Deploy to Streamlit Community Cloud. Write README.

Then Days 23-24: dress rehearsal + Demo Day (Day 25).

THE 10 PROJECT IDEAS

1. Resume vs Job Description Matcher ← Day 14's app!
2. Plagiarism & Similarity Checker
3. Spam Message Classifier
4. Smart Notes Summarizer
5. Campus FAQ Assistant
6. Expense Tracker & Spending Dashboard
7. Habit & Study Streak Tracker
8. Attendance Safety Calculator
9. Blood Donor & Volunteer Finder
10. Movie or Book Recommender

Pick ONE. You can also bring your own idea — as long as I approve the scope.

TOMORROW

Groups announced. Projects picked. GitHub repos created. The build begins. Sleep well tonight.

15 days of learning. 10 days of building. 1 demo.

Everything you've learned comes together in the group project. Tomorrow. See you at 9.

YOUR TOOLKIT

- ✓ **Python** — functions, files, conditionals
- ✓ **Streamlit** — widgets, layout, session_state, forms
- ✓ **pandas** — DataFrames, filtering, charts
- ✓ **SQLite** — CREATE, INSERT, SELECT, JOIN
- ✓ **Regex** — findall, search, sub, patterns
- ✓ **PDF/Word** — PyPDF2, python-docx
- ✓ **APIs** — requests, JSON, caching, errors
- ✓ **TF-IDF** — text similarity, cosine, scikit-learn
- ✓ **Git** — commit, push, pull, GitHub

WHERE YOU'VE BEEN

- Days 1-3:** Tools (Python, Streamlit, Git)
- Days 4-8:** Building (widgets, data, charts, solo challenge)
- Days 9-15:** Real skills (databases, text, APIs, AI, recap)
- Days 16-22:** Group project (5 sprints)
- Days 23-25:** Ship & present

📍 You are HERE → Day 15. The last day of learning.

THE FINAL PUSH

- 👥 **Day 16** — Groups + project kickoff
- ⏪ ⏩ **Days 17-21** — 5 sprints of building
- 📦 **Day 22** — Deploy to Streamlit Cloud
- 🎭 **Day 23** — Dress rehearsal
- 🚀 **Day 25** — Demo Day. Certificates.